

Reviewing the GNUSim 8085

This article reviews the GNUSim8085—a graphical simulator, assembler and debugger for the Intel 8085 microprocessor.

The Intel 8085 is an 8-bit microprocessor that was launched by Intel in 1977 and hence is something that we have never seen around. So why do we need to learn about the programming model and instruction set of this old microprocessor? The microprocessors we use are all for general-purpose computing but there are other applications of computers such as making automatic solar-tracking-panels, automatic power-controls, or security control systems. For such applications, general-purpose CPUs are unnecessary—a waste of resources; a low-powered CPU is what we need. The Intel 8085 is one of the candidates.

To program this piece of hardware, we first need to know the programming model—the logical structure of the programmable registers, flags, and the instruction set. The Intel 8085 has seven internal general-purpose 8-bit registers A, B, C, D, E, H, L and 5 flags – S (sign), Z (zero), AC (Aux Carry), P (Parity) and CY (Carry). The BC, DE and HL registers can be used as register pairs and work as 16-bit registers, with some specific instructions. Besides these, there are two other registers: one is the program counter (PC), which is a 16-bit register, and always points to the memory address of the next instruction to be processed; the other is the stack pointer (SP), a 16-bit register that points to the top of the stack.

The processor has a total of 246 instructions with which we can manipulate data in the processor registers and memory, and also perform conditional branching in code with the help of flag values, which are set by the instructions, depending on the result. This is a brief overview of the programming model; see the *References* for links to more information.

Programming the Intel 8085 or similar microprocessors

is not like present-day programming—there is no editor, compiler or assembler, and not even a monitor. You can probably have a 7-segment display, which has to be manually interfaced, to show the memory content. You need to burn in the assembled program (essentially, some numbers in memory), connect the memory with the processor, and the microprocessor starts to run the code. There are programmers to burn the code into memory, and microprocessor kits that help you program the microprocessor interactively by providing additional controllers, 7-segment displays, a 4x4 keyboard, etc. However, you still have to hand-assemble the code.

Now, no code is perfect the first time; that's why we have bugs. Debugging code on an 8085 itself is a real pain. Even if you use a microprocessor kit with a debugging facility, the limited debugging features make it a complex and slow process, not suited for a beginner. So, essentially, to know if your code is correct or not, completely depends on your skills, the expected outputs of the code, and the minimal debugging interface.

Wouldn't it be great to be able to test the code and develop it completely before even touching the actual processor? This is what a simulator lets you do – the very purpose of the GNUSim8085 Intel 8085 Microprocessor Simulator.

About the simulator

The 8085 microprocessor simulator simulates the Intel 8085 programming model, with the same instructions available as on the real system. The input and the output of the actual machine and the simulator are identical, although the internal processing method may differ. The simulator lets

